Multilist and Skiplist

Concept of ADT : Revision :

So far, we have learnt two data structures list and stack. We have realized that these data structures can be implemented in different ways – but they support the interface. Basically, we have created a new type. Once we create them, we can use them like any other built-in type. So, we have enhanced the language.

We have realized that separating the concerns leads to a good software development. A data structure has number of components. A data structure exhibits certain behaviour independent of the component. Each component has its own behaviour which does not depend on the data structure where it is included.

A polynomial has number of terms. A term has coefficient and exponent. We could represent the polynomial using an array of terms or list of terms. Term should be independent of the data structure which contains it. Therefore making the term self-referential is wrong. Think of an array of terms. If it has a pointer to itself, what can we put there?

Similarly if we implement the polynomial using a list, the list should know nothing of the term. The list should provide way to initialize, traverse, insert and so on. Whenever any behaviour of the component is required, it should invoke the suitable function of the component.

Trust these points are clear. We will try to emphasize this throughout the course.

Skiplist :

Let us consider that I want to travel to a village between Srirangapatna and Mysore. We may have shuttle buses from Bangalore to that village. It will stop at every village. It will take lot of time to reach my destination.

Can we take an express bus to Mysore and then come back?

Can we take a direct bus to Srirangapatna which does not stop at every village and then take a local bus to the village from Srirangapatna?

Yes. Definitely. This will save time.

This concept is called a skip list.

Limitation of a normal list:

Let us consider an example of keys arranged in non decreasing order.

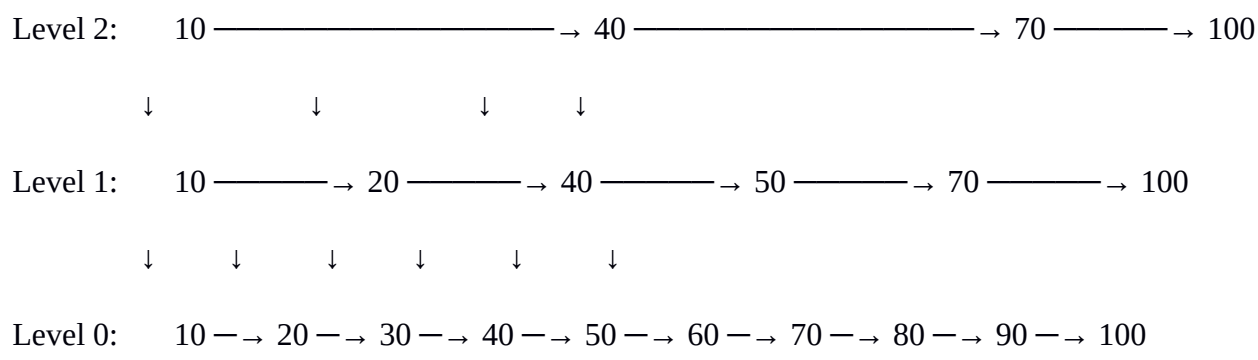
10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100 → NULL

Searching for 90 would require that we start from the first node and traverse almost the whole length of the list.

In the worst case, the number of steps and comparisons will be proportional to the number of elements in the list.

Structure of skiplist :

Can we arrange this in multiple levels?



In level2, we start at the node having key 10 and then jump to the node having key 40 and then 70 and then 100. We cover the whole list in a few steps – skip part of the list in each jump.

In level1, we access more nodes while traversing compared to level 2.

In level0, we walk through the list one step at a time.

Searching in a skiplist :

Let us say that I am searching for 90.

We start level 2.

Level 2: 10 ——————→ 40 ——————→ 70 ——————→ 100

↓ ↓

Level 1: 10 ——→ 20 ——→ 40 ——→ 50 ——→ 70 ——→ 100

↓ ↓

Level 0: 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100

Step 1

Start at 10 at the highest level.

10 ——————→ 40 ——————→ 70 ——————→ 100

^

10 < 90, so continue

Step 2

Move to 40.

40 < 90, so continue.

Step 3

Move to 70.

70 < 90, so continue.

Step 4

The next node is 100.

But:

100 > 90

So we will not find 90 here..

We drop down a level at 70.

Level 2: 10 ————→ 40 ————→ 70 ————→ 100

↓

Level 1: 10 —→ 20 —→ 40 —→ 50 —→ 70 —→ 100

Level 0: 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100

Now at Level 0:

70 → 80 → 90

We reach 90.

So instead of examining all nine elements, the higher levels allowed us to **jump over large portions of the list**.

Structure of nodes and other operations on the skiplist:

A skip list may have any number of levels. The number of levels depends on the expected number of nodes in the list. Normally, each higher level will have half the nodes of the next level.

Each node will have pointers for each level. Only those pointers which participate in the list at that level shall be populated.

Insertion, deletion will again follow the concept we discussed in searching. We always start from top and locate all the levels where the new node has to be inserted or deleted.

Note: Implementing a skiplist is not part of the syllabus.

Applications:

- In Memory ordered data for faster insertion, deletion and searching
- Concurrent data structures where operations can happen in parallel
- Priority ordered events

  The skiplists can be used instead of balanced search trees.

Salient points:

A skip list is a sorted linked list augmented with multiple levels of forward pointers. Higher levels allow searches to skip over many elements**.**

1. Bottom level contains all elements.
2. Higher levels contain fewer elements.
3. Higher levels provide shortcuts.

MultiLists:

This means a list of lists. It is possible that a node may be part of more than one list. It is also likely that a list has nodes whose keys are heads of some other list.

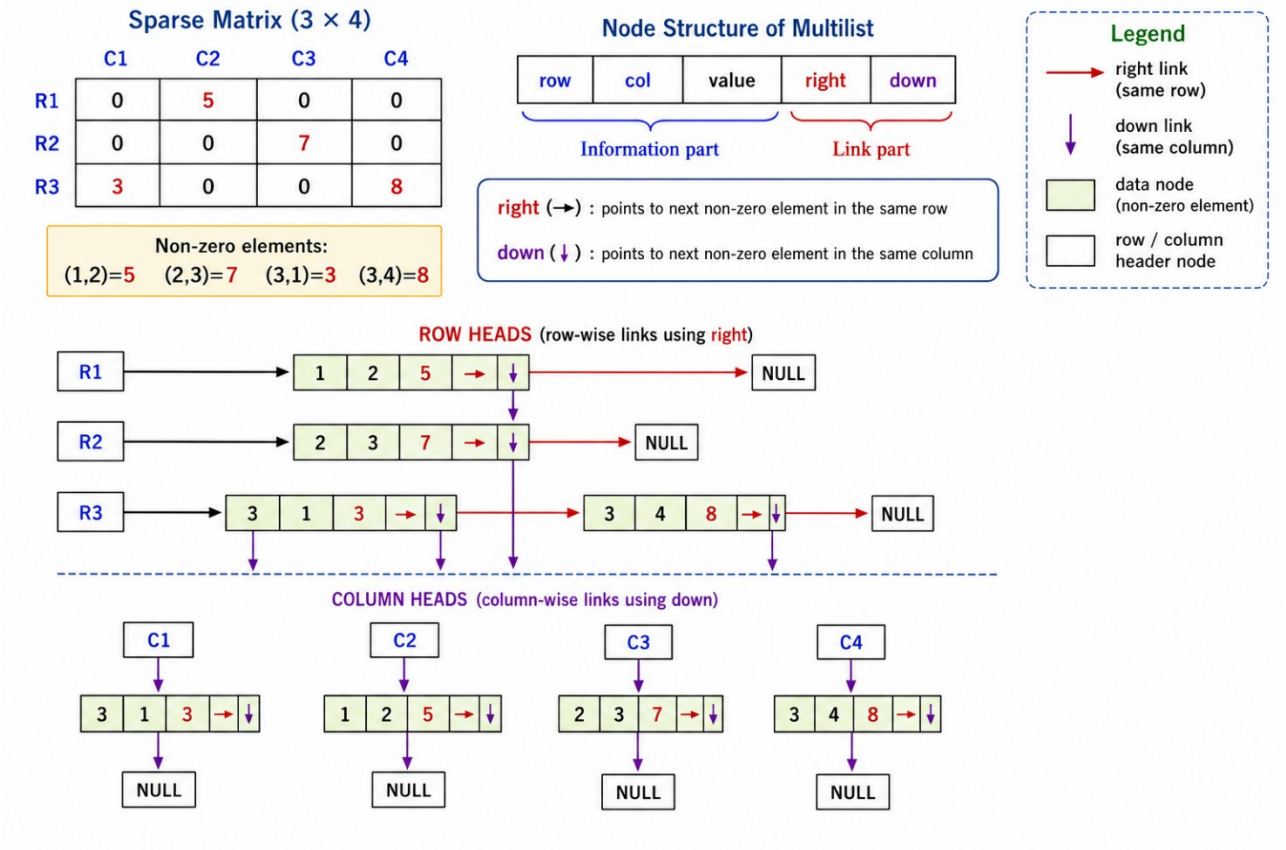
Sparse matrix : time space trade off.

Let us say that we have a huge matrix – 10000 X 10000 elements. These are not uncommon in images. This also happens in techniques called finite element methods or finite differences required in Engineering Applications. This also manifests in Electrical Engineering having huge networks.

These matrices would require a lot of memory. But most of the elements are zero. Can we store only non zero elements? In that case, how to identify the row and the column to which the element belongs to? Can we store the value, the row number and the column number? It is fine, how to traverse rowwise and/or columnwise? Certain operations like addition and multiplication of matrices would require such traversals.

How about making a rowlist for each row which connects every element in that row and a column list for each column which connects every element in that column? Do you observe that in such a case, each element would occur in two lists?

Such a list is called a multilist. It is a list of lists. One possibility is that in such a case, the element occurs in more than one list.



The diagram above depicts how a sparse matrix would store only nonzero values. For clarity, the same node is shown separately in a row list as well as a column list. In practice, there will be only one node for each non zero value.

Row heads and column heads may themselves be stored in a list or an array.

The same node can be reached through different linked lists, each representing a different relationship**.**

This example illustrates how we can gain in memory at the cost of time. Another similar example is file compression.

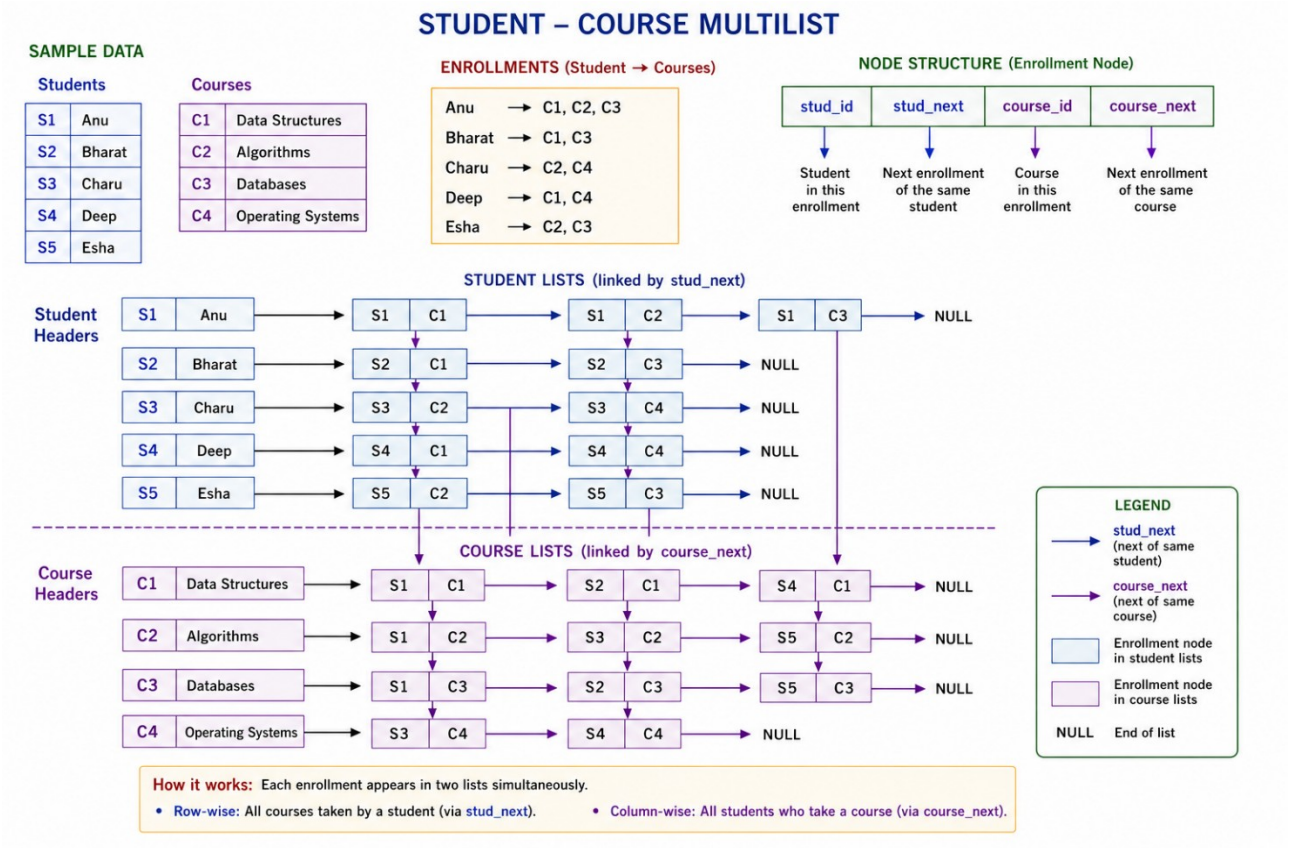
One physical node, multiple logical views:

Let us take an example of students taking a few courses and each course having a few students. It may be necessary to find the courses each student has taken or find the students who have taken a particular course.

One possible solution would be to keep two data structures. Map each student to the list of courses taken by him. Map each course to a list of students. What are the issues in this approach?

This would require more memory. Any change would require updation at multiple places. If the data is not updated at each occurrence, integrity is lost.

The solution would be store the data of student and his course at only one place and allow for traversal student wise or subject wise. This requires a multilist.



In this diagram, the nodes are shown separately. But there will be only one node for each student, subject mapping.

Applications of multilists:

1. Sparse matrix
2. Graph representation
3. Social networks
4. Multiple views as in files.

Salient points :

- Multiple relations
- Avoids duplicates
- Avoids multiple updation
- Complicated pointer management
- Complicated insertion/deletion